

MovFlix

Movie Recommender System

End-to-End GCP Architecture Design

Architecture & Design document
by Richardy Lobo' Sapan

Overview

MovFlix is a movie streaming platform that serves personalised movie recommendations to users through their account dashboard. This document describes an end-to-end GCP-based architecture capable of refreshing recommendations for all users at least daily, while also supporting real-time updates when a user submits a new rating.

The architecture follows a layered pipeline:

- 1. Sources
- 2. Ingestion (GCS + Pub/Sub)
- 3. Processing (Dataflow + Composer)
- 4. Storage (BigQuery + Feature Store)
- 5. ML Training & Serving (Vertex AI)
- 6. Recommendation Serving (Redis + Cloud Run + MovFlix App)

Dataset Overview

The system is seeded with the MovieLens ml-latest-small dataset. In production, MovFlix's own user interaction data grows beyond this seed over time.

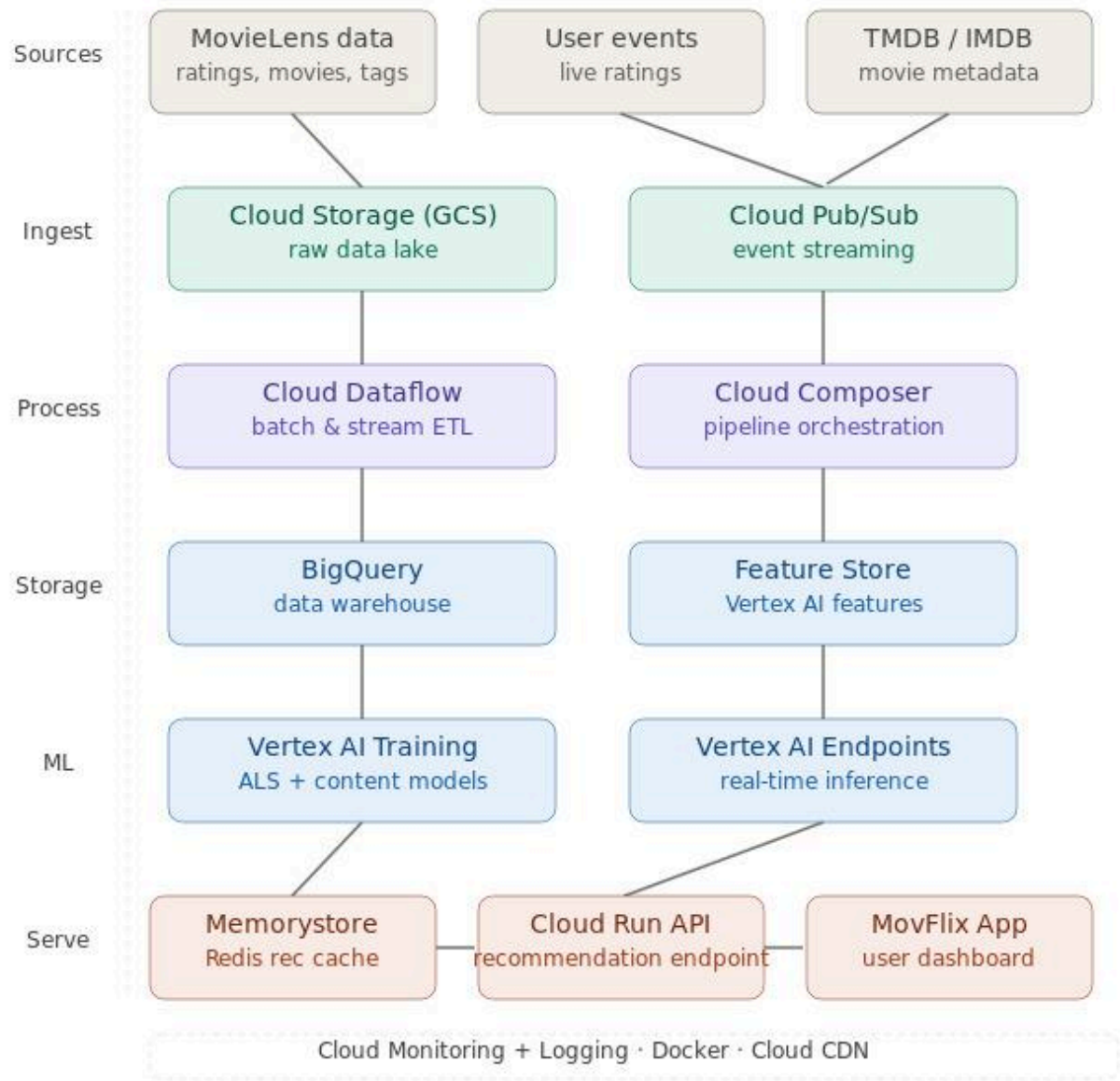
File	Rows	Key Fields
ratings.csv	100,836	userId, movieId, rating (0.5–5.0), timestamp
movies.csv	9,742	movieId, title, genres (pipe-separated, 20 genres)
tags.csv	3,683	userId, movieId, tag (free-text), timestamp
links.csv	9,742	movieId, imdbId, tmdbId (8 nulls)

Key Data Characteristics

- 610 unique users · 9,742 movies · 98.3% sparse ratings matrix
- Average 165 ratings per user, 10 ratings per movie
- Data spans March 1996 to September 2018
- Tags are user-generated free text, valuable NLP signal for content-based filtering
- links.csv enables metadata enrichment via TMDB and IMDB APIs
- 34 movies have no genre listed, cold start edge case requiring handling

Architecture

See the diagram below for the full visual. The six functional layers are described in detail below.



Layer 1: Sources

Component	Description
MovieLens data	Historical batch files: ratings.csv, movies.csv, tags.csv, links.csv used as the cold-start training seed
User events	Real-time rating and interaction events emitted by the MovFlix frontend as users rate movies
TMDB / IMDB API	External movie metadata (posters, descriptions, cast) fetched via IDs from links.csv

Note: The MovieLens dataset is the cold-start seed. Once MovFlix is live, the user events stream becomes the primary training signal and grows to replace it.

Layer 2: Ingestion

Component	Role	Pros	Cons
Cloud Storage (GCS)	Raw data lake for batch files CSVs, nightly TMDB API dumps	Cheap, durable, native GCP integration across all downstream services	Not directly queryable; requires Dataflow or BigQuery load step
Cloud Pub/Sub	Streaming message queue for real-time user rating events	Fully managed; scales to millions of events/sec; at-least-once delivery guarantee	Ordering complexity; requires deduplication logic downstream

TMDB metadata is batch-fetched nightly via a Cloud Composer DAG calling the TMDB REST API, written to GCS. It does not flow through Pub/Sub.

Layer 3: Processing

Component	Role	Pros	Cons
Cloud Dataflow	Apache Beam-based batch ETL (GCS → BigQuery) and stream ETL (Pub/Sub → BigQuery) in a unified model	Handles batch and stream with same code; serverless auto-scaling; strong GCP integration	Higher latency vs Flink for micro-batches; cost spikes on large one-off jobs
Cloud Composer	Apache Airflow-based orchestration of the entire daily batch pipeline — retraining DAG, scoring DAG, Redis refresh	Familiar Airflow interface; managed; rich GCP operator library	Heavyweight for simple pipelines; Composer cluster runs continuously, fixed baseline cost

Daily Batch DAG (triggered by Composer at 02:00 WIB)

1. Ingest new ratings/events from GCS append table
2. Run Dataflow ETL to clean, deduplicate, and transform
3. Load processed data to BigQuery
4. Compute user and movie feature vectors, write to Vertex AI Feature Store
5. Trigger Vertex AI Training job (ALS + content model)
6. Register new model version in Vertex AI Model Registry
7. Run batch scoring, compute top-N recs for all users
8. Bulk-write precomputed recommendations to Memorystore (Redis)
9. Optionally deploy updated model to Vertex AI Endpoints

Layer 4: Storage

Component	Role	Pros	Cons
BigQuery	Central data warehouse: ratings history, user profiles, movie metadata, precomputed feature tables	Serverless; petabyte-scale; SQL interface; native BQML for in-warehouse model experiments	Not suitable for low-latency serving; frequent query costs add up
Vertex AI Feature Store	Stores engineered user and movie embedding vectors for both online reads and batch training consistency	Consistent feature values between training and serving pipelines; low-latency online reads	Setup and versioning overhead; operational complexity vs plain BigQuery

Key BigQuery Tables

- **ratings**: Append-only fact table (userId, movieId, rating, timestamp)
- **movies**: Movie metadata with genre boolean flags
- **user_features**: Precomputed ALS user latent vectors, refreshed daily
- **movie_features**: Precomputed ALS movie latent vectors, refreshed daily
- **precomputed_rec**s: Top-N recommendations per userId, refreshed daily

Layer 5: ML

Component	Role	Pros	Cons
Vertex AI Training	Trains the recommendation model on the latest ratings data	Managed infrastructure; GPU/TPU support; integrates natively with Feature Store and Model Registry	Batch-only by default; cold start spin-up latency per job
Vertex AI Endpoints	Real-time inference endpoint for on-demand scoring when precomputed recs are unavailable	Low-latency serving; auto-scaling; supports A/B testing between model versions	Per-request cost; overkill if batch precomputation covers all use cases

Recommendation Algorithm

1. Primary model: ALS (Alternating Least Squares) Collaborative Filtering

ALS is chosen specifically because the ratings matrix is 98.3% sparse. It decomposes the user-movie interaction matrix into user and movie latent factor vectors, well-suited for explicit 0.5–5.0 star ratings. Implemented as a Vertex AI custom training container using Spark MLlib ALS or implicit library.

Core idea: Users who agreed in the past will agree in the future.

ALS works by factorizing the sparse ratings matrix **R** into two smaller dense matrices:

$$R (610 \times 9742) \approx U (610 \times k) \times M (k \times 9742)$$

U = user latent factor matrix (each user becomes a k-dimensional vector)

M = movie latent factor matrix (each movie becomes a k-dimensional vector)

k = number of latent factors

Those **k latent factors** don't have predefined meanings, the model learns them. But intuitively they end up capturing things like *"how much does this user like action movies"* or *"how artsy is this film"* without anyone labeling them.

How ALS solves it:

It alternates between two steps, fixing one matrix while solving for the other (hence *Alternating* Least Squares):

Step 1: Fix M, solve for each user vector in U

(minimize squared error between predicted and actual ratings)

Step 2: Fix U, solve for each movie vector in M

(same objective)

Repeat until convergence.

Each step is a simple least squares problem, fast and parallelizable across Spark workers on Dataproc/Vertex AI.

Predicting a rating:

$$\text{predicted_rating}(\text{user } u, \text{movie } m) = \text{dot_product}(U[u], M[m])$$

2. Secondary model: Content-Based Filtering (Hybrid)

Uses genre flags (20 genres per movie) and user-generated tags (TF-IDF or Word2Vec) to compute movie content similarity vectors. Addresses two cold start scenarios:

- New movies with no ratings can be recommended via genre and tag similarity
- New users with few ratings receive genre-based recommendations from onboarding preferences

Core idea: Recommend movies similar to ones the user already liked, based on movie attributes. This uses two signals from the dataset:

Signal 1: Genres (from movies.csv)

Each movie gets a binary vector of its genres:

Toy Story → [Action=0, Adventure=1, Animation=1, Children=1, Comedy=1, Fantasy=1, ...]

Movie similarity = cosine similarity between genre vectors.

Signal 2: Tags (from tags.csv)

Tags are user-generated free text ("atmospheric", "thought-provoking", "Disney"...). Apply TF-IDF across all tags per movie to build a tag vector per movie, then compute similarity.

```
movie_content_vector = concat(genre_vector, tfidf_tag_vector)
similarity(movie_a, movie_b) = cosine_similarity(vector_a, vector_b)
```

User content profile:

Average the content vectors of movies the user rated highly (e.g. ≥ 3.5 stars):

```
user_profile = mean(content_vector[m] for m where rating[u,m] >= 3.5)
```

Recommended movies = highest cosine similarity to **user_profile**, among unseen movies.

Note that neither algorithm alone is complete:

Situation	ALS alone	Content alone
New user (< 5 ratings)	Fails: not enough signal	Works: use genre preference
New movie (< 5 ratings)	Fails: not in training matrix	Works: use genre similarity
User with many ratings	Works well	Can get stuck in a genre bubble
Obscure niche taste	Works if similar users exist	Fails if no matching tags/genres

Hybrid scoring formula:

```
final_score = alpha * ALS_score + (1 - alpha) * content_score

alpha -> 1   for users with many ratings (trust collaborative signal)
alpha -> 0   for new users with few ratings (trust content signal)
```

Layer 6: Serving

Component	Role	Pros	Cons
Memorystore (Redis)	Caches precomputed top-N recommendations per userId for sub-millisecond dashboard reads	Sub-millisecond read latency; ideal for high-concurrency dashboard loads	Volatile, requires daily refresh; memory-bound and cost scales with data size
Cloud Run API	Stateless REST API that reads from Redis and returns recommendations to the frontend	Fully managed; scales to zero when idle; containerised (Docker) for portability	Cold start latency after idle periods; stateless design requires Redis for all state
MovFlix App	User-facing frontend dashboard rendering the recommendation carousel and user account page	Consumes the Cloud Run API; can be hosted on Firebase / Cloud CDN for edge caching	Frontend implementation is outside the scope of this architecture document

Serving Flow

```

User opens dashboard
-> GET /recommendations?userId=X (Cloud Run API)
-> Read precomputed top-N from Redis (< 1ms)
-> Return movie list with metadata
-> Frontend renders recommendation carousel

Fallback (cache miss):
-> Cloud Run calls Vertex AI Endpoints for real-time scoring
-> Score is cached in Redis with TTL = 25h

```

Redis TTL is set to 25 hours so stale recommendations are never served beyond one missed daily refresh cycle.

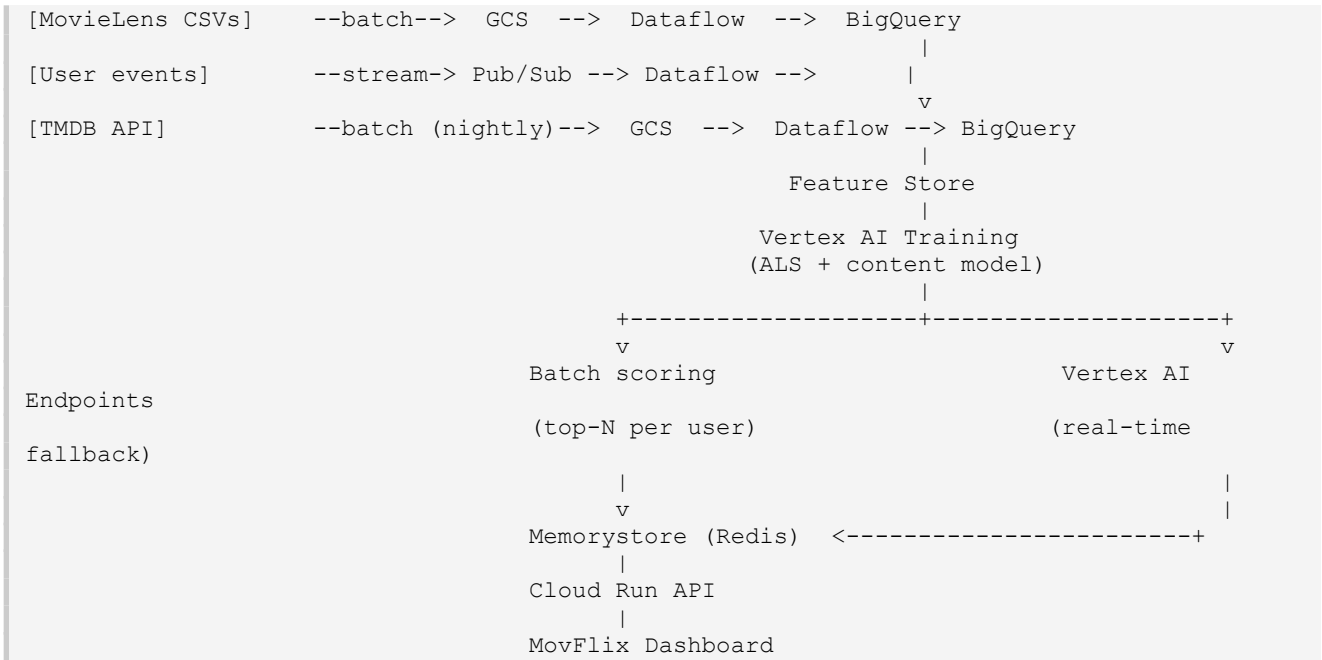
Additional Technologies

Tool	Purpose
Docker	All Dataflow, Cloud Run, and Vertex AI workloads run in containers for reproducibility and portability
Cloud Monitoring + Logging	Pipeline health dashboards, API latency alerting, recommendation freshness monitoring
Cloud CDN	Caches MovFlix frontend static assets and API responses at the edge for reduced latency
Terraform	Infrastructure-as-Code provisioning for all GCP resources, reproducible environments
Vertex AI Model Registry	Tracks model versions, evaluation metrics, and deployment history for auditability

Technology Pros & Cons Summary

Technology	Pros	Cons
Cloud Storage (GCS)	Cheap, durable, native GCP integration	Not queryable; needs processing layer
Cloud Pub/Sub	Scalable, managed, at-least-once delivery	Ordering complexity, dedup overhead
Cloud Dataflow	Unified batch+stream; serverless auto-scaling	Cost spikes; Apache Beam learning curve
Cloud Composer	Managed Airflow; rich DAG ecosystem	Always-on cluster cost
BigQuery	Serverless SQL warehouse; massive scale	Not for low-latency serving
Vertex AI Feature Store	Consistent train/serve features	Setup and versioning complexity
Vertex AI Training	Managed ML; GPU support; Model Registry	Batch-only; spin-up latency per job
Vertex AI Endpoints	Low-latency scalable inference	Per-request cost; overkill if batch suffices
Memorystore (Redis)	Sub-millisecond read latency	Volatile; memory-bound cost
Cloud Run	Serverless containers; scales to zero	Cold start on idle

Data Flow Summary



Assumptions

1. The existing real-time recommender subsystem described in the brief is mapped to Vertex AI Endpoints, which handles online learning and real-time inference.
2. Batch retraining runs daily at minimum. More frequent retraining can be triggered by Pub/Sub message volume thresholds via a Cloud Composer sensor.
3. The MovieLens dataset is the historical seed. In production, MovFlix's own user interaction data grows beyond and eventually replaces it.
4. Cold start for new users is handled via content-based filtering (genre preference from onboarding survey) until sufficient ratings are collected for collaborative filtering.
5. Cold start for new movies is handled via genre and tag similarity until rated by at least 5 users.
6. Redis TTL is set to 25 hours so stale recommendations are never served beyond one missed daily refresh cycle.
7. The 8 missing tmdbIds in links.csv are handled gracefully, movies without TMDB IDs fall back to genre-only content features.

Citation

F. Maxwell Harper and Joseph A. Konstan. 2015. The MovieLens Datasets: History and Context. ACM Transactions on Interactive Intelligent Systems (TiiS) 5, 4: 19:1-19:19.

<https://doi.org/10.1145/2827872>